

Plan maestro actualizado

Bloques 0 a 9: estado real, trabajo terminado, pendientes, evidencias, operación local y puertas de salida.

ESTADO ACTUAL	ALCANCE
Bloques 0 a 4 cerrados	Desarrollo local cerrado
Siguiente: Bloque 5	Netcup: Bloque 8

1. Lectura ejecutiva

Este documento reemplaza la lectura aislada del plan original y del resumen del Bloque 4. El checklist vigente se conserva completo: un bloque puede estar cerrado y aun mostrar deuda transferida a una fase posterior.

5	88/88	0	10
bloques cerrados	pruebas actuales	vulnerabilidades npm	tablas PostgreSQL

Mapa de bloques

Bloque	Estado	Avance checklist	Lectura
0. Línea base y protección de datos	CERRADO	19/20	Cierre técnico; falta coordinación
1. Seguridad crítica	CERRADO	33/33	Seguridad crítica cerrada
2. Esquema reproducible y PostgreSQL	CERRADO CON DEUDA TRANSFERIDA	30/32	Diseño cerrado; deuda transferida
3. Suite de pruebas	CERRADO	21/21	Suite y CI cerradas
4. Migración SQLite a PostgreSQL	CERRADO	17/17	PostgreSQL y Docker cerrados
5. Arquitectura y mantenibilidad	SIGUIENTE	1/13	Trabajo inmediato
6. Funciones e integraciones	PENDIENTE	0/18	Después de arquitectura
7. Interfaz y QA en navegador	PARCIAL	8/20	QA parcial; cierre posterior
8. Staging en Netcup	PENDIENTE	0/14	Staging restringido
9. Demo final cerrada y cierre	PENDIENTE	0/13	Demo y apagado

Decisión actual

Continuar con el Bloque 5. No iniciar Netcup mientras arquitectura, integraciones y QA mantengan puertas abiertas. Todas las pruebas y demostraciones utilizan usuarios y datos ficticios.

2. Entorno operativo y reglas comunes

Docker Desktop en Windows

El flujo recomendado se ejecuta desde PowerShell con contenedores Linux. Docker levanta PostgreSQL, aplica migraciones y después inicia Astro.

```
Copy-Item .env.example .env
docker compose -f compose.dev.yml up -d --build web
docker compose -f compose.dev.yml exec web npm run db:pg:smoke
docker compose -f compose.dev.yml --profile telegram up -d bot
docker compose -f compose.dev.yml --profile scheduler run --rm scheduler
docker compose -f compose.dev.yml --profile telegram --profile scheduler down
```

Reglas de operación

Web	http://127.0.0.1:4321
PostgreSQL	127.0.0.1:5434; nunca se expone públicamente.
Telegram	Solo una instancia de polling por token.
Scheduler	Ejecuta una revisión y termina.
Volúmenes	No usar down -v sin autorización para borrar datos.
Node nativo	Versión >=22.12.0 y <23.0.0.

Secretos e identidad Git

Los secretos viven únicamente en .env y nunca se copian a chats, logs, capturas, commits o CI. El autor esperado es moises <moisesantoniom@gmail.com> y el remoto usa el alias SSH github-work. GitHub CLI no se usa sin confirmar antes la cuenta activa.

Migración y rollback

El importador SQLite -> PostgreSQL hace dry-run de forma predeterminada. El modo commit requiere copia consistente, revisión de conteos y plan de rollback. SQLite permanece intacto y seleccionable con DATABASE_ENGINE=sqlite.

Bloque 0

Línea base y protección de datos

CERRADO

Prioridad: crítica.

Propósito y resultado	Se estableció una línea base reproducible, se protegieron datos locales y se retiró la migración destructiva del flujo habitual.
Evidencia	Node 22.23.1; 25 pruebas iniciales; build SSR aprobado; base inexistente confirmada; migrador seguro incorporado posteriormente.
Pendiente o límite	Queda definir responsables y fecha objetivo de la demostración.
Lectura del checklist	19/20 ítems marcados

Detalle completo del bloque

Repositorio y dependencias

HECHO	Ejecutar <code>npm ci</code> con la versión de Node compatible con el proyecto.
HECHO	Registrar versiones efectivas de Node, npm, Astro, React y SQLite.
HECHO	Ejecutar y guardar el resultado inicial de:
HECHO	<code>npm test</code>
HECHO	<code>npm run lint</code>
HECHO	<code>npm run build</code>
HECHO	Clasificar cada fallo como problema del entorno, del código o de los tests.
HECHO	Revisar que <code>.env.example</code> contenga todas las variables requeridas y únicamente placeholders.
HECHO	Ejecutar <code>npm audit</code> y registrar 5 avisos moderados, 4 altos y 2 críticos.

Base SQLite

HECHO	Localizar cualquier archivo <code>novatareas.db</code> usado por el equipo.
HECHO	Confirmar que no hay una base local que respaldar, inventariar o validar.
PEND.	Si otro integrante entrega una base, crear una copia de solo lectura y registrar su hash, esquema, conteos, integridad y naturaleza ficticia.

Bloquear el riesgo de migraciones

HECHO	Retirar <code>002_new_user_schema.cjs</code> del comando normal de migración.
HECHO	Evitar que cualquier arranque ejecute migraciones destructivas.
HECHO	Añadir una protección temporal que rechazara el comando normal de migración.
HECHO	Reemplazarla por un migrador seguro, transaccional e idempotente.

Puerta de salida

HECHO

Dependencias instalables de forma reproducible con Node 22.12 o posterior dentro de la línea 22.

HECHO

Resultados iniciales de test, lint y build registrados.

HECHO

Base existente respaldada o confirmación de que no hay datos que conservar.

HECHO

Ningún comando habitual puede borrar usuarios o tareas.

Bloque 1

Seguridad crítica

CERRADO

Prioridad: crítica. Puede avanzar en paralelo con la preparación de tests.

Propósito y resultado	Se cerraron vulnerabilidades críticas, recuperación, sesiones, ownership, XSS, validación de entradas e integraciones protegidas.
Evidencia	Astro 7.1.3; Vitest 4.1.10; 55 pruebas; build aprobado; npm audit pasó de 11 avisos a 0; QA real en navegador.
Pendiente o límite	El rate limiting sigue en memoria y no soporta varias instancias.
Lectura del checklist	33/33 ítems marcados

Detalle completo del bloque

Dependencias vulnerables

HECHO

Investigar las guías oficiales compatibles para migrar Astro 4 a una versión corregida, junto con @astrojs/node y @astrojs/react.

HECHO

Actualizar Vitest y @vitest/coverage-v8 de forma controlada.

HECHO

Ejecutar npm audit, tests, lint, build y pruebas de navegador después de cada salto mayor.

HECHO

Mantener el servidor de desarrollo limitado a localhost.

Recuperación y sesión

HECHO

Eliminar toda eliminación automática de cuentas por respuestas fallidas.

HECHO

Sustituir Math.random() por tokens generados con node:crypto.

HECHO

Hacer los tokens de recuperación de un solo uso y con expiración.

HECHO

Aplicar rate limiting a recuperación y login.

HECHO

Guardar respuestas de seguridad con hash o sustituirlas por un mecanismo más seguro para la demo.

HECHO

Configurar cookies con HttpOnly, Secure y SameSite.

HECHO

Invaldar sesiones anteriores después de cambiar la contraseña.

XSS y validación de inputs

HECHO

Eliminar innerHTML cuando contenga títulos, historial, comentarios o respuestas de IA.

HECHO

Eliminar handlers onclick contruidos con texto del usuario.

- | | |
|-------|--|
| HECHO | Renderizar datos de tareas con escape o <code>textContent</code> . |
| HECHO | Validar título, descripción, etiqueta, prioridad, estado y fecha en el servidor. |
| HECHO | Limitar tamaños de campos de tareas y comentarios. |
| HECHO | Validar contenido real y tamaño de avatares, no solo su extensión. |

Rutas e integraciones

- | | |
|-------|--|
| HECHO | Hacer obligatorio <code>CRON_SECRET</code> y fallar de forma cerrada si falta. |
| HECHO | Validar el secreto del webhook de Telegram. |
| HECHO | Escapar texto antes de enviarlo con <code>parse_mode</code> : HTML a Telegram. |
| HECHO | Añadir y validar <code>state</code> en Google OAuth. |
| HECHO | Evitar que tokens o claves aparezcan en respuestas y logs. |
| HECHO | Proteger la API inteligente externa con una <code>AI_API_KEY</code> independiente de la clave privada del proveedor <code>ZAI_API_KEY</code> . |

Tests de seguridad

- | | |
|-------|--|
| HECHO | Respuestas de recuperación incorrectas no eliminan una cuenta. |
| HECHO | Un usuario no puede leer ni modificar tareas de otro. |
| HECHO | Cron sin secreto devuelve 401. |
| HECHO | Webhook falsificado es rechazado. |
| HECHO | Payloads con HTML y scripts se validan y se muestran como texto en el flujo probado. |
| HECHO | <code>state</code> de Google alterado o ausente es rechazado por el verificador. |

Puerta de salida

- | | |
|-------|---|
| HECHO | No quedan rutas conocidas que permitan borrar cuentas externamente. |
| HECHO | Ownership validado en todas las rutas de tareas, subtareas e historial. |
| HECHO | Inputs externos se validan antes de escribir en la base. |
| HECHO | Tests negativos de seguridad aprobados. |

Bloque 2

Esquema reproducible y PostgreSQL

CERRADO CON DEUDA TRANSFERIDA

Prioridad: alta. No migrar datos antes de cerrar el diseño.

Propósito y resultado	Se diseñó el esquema reproducible para PostgreSQL 16 y SQLite, con tipos, restricciones, cifrado de tokens y repositorios iniciales.
Evidencia	10 tablas; 115 restricciones; 65 pruebas; build y audit aprobados; QA en 390 x 844 y 1280 x 900.
Pendiente o límite	Parte del SQL heredado aún debe salir de páginas y scripts. Las transacciones PostgreSQL previstas se completaron durante el Bloque 4.
Lectura del checklist	30/32 ítems marcados

Detalle completo del bloque

Diseño

HECHO	Adoptar Drizzle ORM y Drizzle Kit, salvo que el equipo documente otra decisión.
HECHO	Crear una migración inicial SQLite completa para una base vacía.
HECHO	Definir todas las tablas actuales en SQLite:
HECHO	users
HECHO	security_questions
HECHO	categories
HECHO	tasks
HECHO	subtasks
HECHO	task_history
HECHO	task_comments
HECHO	task_embeddings
HECHO	Crear una tabla separada para recomendaciones de IA.
HECHO	Definir columnas actuales de Google Calendar en el esquema SQLite.
HECHO	Crear un registro real de migraciones aplicadas.

Tipos y restricciones

HECHO	Correo NOT NULL, único y sin distinción de mayúsculas en SQLite.
HECHO	Estados y prioridades con valores permitidos.
HECHO	DATE para fecha límite sin hora.

HECHO	TIMESTAMPZ para recordatorios con instante específico.
HECHO	Timestamps de auditoría con zona horaria.
HECHO	Claves foráneas y políticas ON DELETE explícitas en SQLite.
HECHO	Índices por usuario, estado, prioridad, fecha y archivado en SQLite.
HECHO	Cifrar tokens persistidos de Google.

Acceso a datos

HECHO	Crear repositorios PostgreSQL iniciales para usuarios, tareas y comentarios.
HECHO	Retirar el acceso SQLite duplicado del bot y compartir el módulo actual.
HECHO	Inventariar las transformaciones específicas de SQLite y documentar su equivalente PostgreSQL.
PEND.	Retirar el SQL restante de páginas y scripts durante el Bloque 4.
PEND.	Ejecutar escrituras relacionadas dentro de transacciones PostgreSQL durante el Bloque 4.

Puerta de salida

HECHO	Una base PostgreSQL embebida vacía se crea y verifica con <code>npm run db:pg:verify</code> .
HECHO	Repetir las migraciones SQLite actuales no elimina ni duplica datos.
HECHO	Diagrama y diccionario de datos actualizados.
HECHO	Tests del esquema y las migraciones SQLite actuales aprobados.
HECHO	Tests del esquema y la migración PostgreSQL aprobados con PGLite. La repetición contra un servicio PostgreSQL 16 persistente es la primera precondition del Bloque 4 cuando Docker o WSL estén disponibles.

Bloque 3	Suite de pruebas	CERRADO
----------	-------------------------	---------

Prioridad: alta. Debe crecer junto con cada corrección.

Propósito y resultado	Se amplió la suite determinista y se configuró integración continua con PostgreSQL 16 efímero, cobertura y artefactos.
Evidencia	80 pruebas al cierre; GitHub Actions verde en testing; migraciones, cobertura y build aprobados sin servicios externos reales.
Pendiente o límite	La evidencia posterior del Bloque 4 elevó la suite a 88 pruebas.
Lectura del checklist	21/21 ítems marcados

Detalle completo del bloque

Pruebas automáticas

HECHO	Registro, login, logout y cambio de contraseña.
HECHO	Recuperación de contraseña y rate limiting.

HECHO	CRUD completo de tareas.
HECHO	Completar, reabrir, archivar y desarchivar.
HECHO	Ownership entre dos usuarios ficticios.
HECHO	Subtareas, historial y comentarios.
HECHO	Fechas vencidas, próximas y zona America/El_Salvador.
HECHO	Recordatorios enviados una sola vez.
HECHO	Telegram, cron y Google con servicios simulados.
HECHO	z.ai disponible, Ollama disponible y fallback sin red.
HECHO	Creación y reejecución de migraciones.

CI

HECHO	Configurar npm ci, lint, tests y build en GitHub Actions.
HECHO	Configurar una base PostgreSQL 16 efímera para tests de integración.
HECHO	No utilizar claves reales ni consumir saldo de IA en CI.
HECHO	Guardar el reporte de cobertura como artefacto durante 14 días, sin perseguir un porcentaje artificial.
HECHO	Bloquear integración cuando falle una verificación crítica.
HECHO	Confirmar la primera ejecución verde en GitHub Actions sobre testing.

Puerta de salida

HECHO	Flujos críticos deterministas en local y workflow de CI definido.
HECHO	Tests no dependen de servicios externos reales.
HECHO	Fallos muestran información suficiente para reproducirse.
HECHO	Ejecución remota del workflow confirmada en verde.

Bloque 4

Migración SQLite a PostgreSQL

CERRADO

Prioridad: alta. Solo aplica si existe información que conservar.

Propósito y resultado	PostgreSQL pasó a ser el runtime principal en Docker. La web, el bot y el scheduler comparten un adaptador asíncrono con SQLite como rollback.
Evidencia	88 pruebas; importación repetida dos veces; conteos equivalentes; cero filas huérfanas; login y tareas aprobados; QA Docker en escritorio y móvil.
Pendiente o límite	La operación en Netcup sigue fuera de alcance.
Lectura del checklist	17/17 ítems marcados

Detalle completo del bloque

Tareas del bloque

HECHO	Completar el acceso de lectura y escritura para todas las rutas mediante un adaptador asíncrono compatible con SQLite y PostgreSQL.
HECHO	Sustituir GROUP_CONCAT, datetime('now'), unixepoch(), booleanos 0/1 y la obtención SQLite del último ID.
HECHO	Ejecutar escrituras relacionadas dentro de transacciones PostgreSQL.
HECHO	Crear exportador de SQLite en modo solo lectura.
HECHO	Conservar identificadores originales cuando sea posible.
HECHO	Transformar fechas, booleanos, correos y estados.
HECHO	Importar tablas en orden de dependencias y dentro de transacciones.
HECHO	Comparar conteos por tabla antes y después.
HECHO	Verificar claves foráneas y filas huérfanas.
HECHO	Verificar hashes de contraseña mediante login, sin exponerlos.
HECHO	Comparar muestras funcionales por usuario ficticio.
HECHO	Ensayar la migración al menos dos veces desde una copia limpia.
HECHO	Mantener SQLite intacto hasta aceptar PostgreSQL.
HECHO	Documentar rollback.

Puerta de salida

HECHO	Conteos y relaciones equivalentes.
HECHO	Login y tareas funcionan sobre PostgreSQL.
HECHO	Migración y rollback reproducibles.

Bloque 5

Arquitectura y mantenibilidad

SIGUIENTE

Prioridad: media-alta.

Propósito y resultado	Consolidar arquitectura y mantenibilidad antes de ampliar funciones o preparar un servidor.
Evidencia	El modelo z.ai predeterminado ya fue alineado a glm-4.5-flash.
Pendiente o límite	Prioridad inmediata: IA unificada, separación de subtareas, modularización del dashboard, limpieza Supabase y estado explícito del bot.
Lectura del checklist	1/13 ítems marcados

Detalle completo del bloque

Tareas del bloque

PEND.	Unificar proveedor, modelo, prompts, timeouts y fallbacks de IA.
PEND.	Registrar fuente de cada recomendación: z.ai, Ollama, historial o reglas.
PEND.	Separar recomendaciones de IA de las subtareas reales.
PEND.	Dividir gradualmente dashboard.astro en módulos y componentes.
PEND.	Separar lógica de presentación, llamadas API y estado del calendario.
PEND.	Retirar módulos muertos de la migración abandonada a Supabase.
PEND.	Eliminar configuración y nombres heredados de Gemini que ya no apliquen.
HECHO	Corregir discrepancias del modelo z.ai predeterminado.
PEND.	Definir claramente los procesos web, bot y scheduler.
PEND.	Persistir o manejar de forma explícita el estado conversacional del bot.

Puerta de salida

PEND.	Una regla de negocio se implementa una sola vez.
PEND.	Web, API y Telegram producen comportamientos coherentes.
PEND.	El dashboard puede modificarse sin tocar un archivo monolítico.

Bloque 6

Funciones e integraciones

PENDIENTE

Prioridad: media.

Propósito y resultado	Completar integraciones funcionales de Telegram, recordatorios, Google Calendar, IA/RAG y almacenamiento de avatares.
Evidencia	Existen QA parciales de Telegram y z.ai con datos ficticios.
Pendiente o límite	No declarar recordatorios completos hasta definir y persistir reminder_at.
Lectura del checklist	0/18 ítems marcados

Detalle completo del bloque

Telegram y recordatorios

PEND.	Corregir el flujo de errores al crear tareas desde Telegram.
PEND.	Validar que las categorías pertenezcan al usuario vinculado.
PEND.	Conservar o recuperar sesiones del bot tras reinicios.
PEND.	Separar fecha límite de fecha y hora de recordatorio.
PEND.	Reprogramar avisos cuando cambie la fecha.
PEND.	Verificar que avisos próximos y vencidos no se repitan.

Google Calendar

PEND.	Completar conexión, consulta y desconexión.
PEND.	Manejar expiración y renovación de tokens.
PEND.	Mostrar estados de conexión y errores en el dashboard.
PEND.	Probar rangos de fechas y eventos de día completo.

IA y RAG

PEND.	Indexar historial de forma consistente.
PEND.	Reindexar cuando una tarea archivada cambie.
PEND.	Validar dimensiones y modelo de embeddings.
PEND.	Limitar costo y frecuencia por usuario.
PEND.	Mostrar cuándo se utilizó un fallback.

Archivos y avatares

PEND.	Evitar acumulación de avatares antiguos.
PEND.	Preparar una abstracción de almacenamiento persistente.

PEND.

Probar contenido inválido, archivos grandes y nombres maliciosos.

Bloque 7

Interfaz y QA en navegador

PARCIAL

Prioridad: media.

Propósito y resultado	Cerrar QA funcional, responsivo y accesible en navegador sobre los flujos principales.
Evidencia	Ya se corrigieron registro, logout, archivo, filtros, agenda, recuperación, vinculación Telegram y logs visibles.
Pendiente o límite	Faltan teclado, foco, contraste, estados de error, textos largos y matrices completas de móvil/escritorio.
Lectura del checklist	8/20 ítems marcados

Detalle completo del bloque

Tareas del bloque

HECHO	Registro crea la cuenta e inicia la sesión sin devolver silenciosamente al login.
HECHO	Logout reemplaza el historial y las páginas autenticadas desactivan caché.
HECHO	Reabrir una tarea archivada funciona desde la vista Archivo.
HECHO	Crear una tarea limpia filtros de prioridad y etiqueta que podrían ocultarla.
HECHO	La agenda muestra únicamente los eventos del mes seleccionado.
HECHO	Recuperación no revela si el correo existe y permite volver al login.
HECHO	Telegram usa códigos temporales de un solo uso en vez de pedir contraseña.
PEND.	Probar registro y login en escritorio y móvil.
PEND.	Probar listas vacías, muchas tareas y textos largos.
PEND.	Probar modales, Escape, foco inicial y restauración de foco.
PEND.	Probar calendario, cambio de mes y eventos de Google.
PEND.	Probar mensajes de error cuando la API falla.
PEND.	Revisar navegación completa con teclado.
PEND.	Revisar contraste, labels y nombres accesibles.
PEND.	Revisar anchos móviles representativos y escritorio.
HECHO	Eliminar logs de depuración visibles del dashboard.
PEND.	Ejecutar un smoke test completo después de cada cambio importante.

Puerta de salida

PEND.

No hay bloqueos funcionales en móvil o escritorio.

PEND.

Los errores son visibles y recuperables.

PEND.

Los flujos principales funcionan con teclado.

Bloque 8

Staging en Netcup

PENDIENTE

Prioridad: después de cerrar los riesgos críticos.

Propósito y resultado	Crear staging temporal y restringido en Netcup una vez cerrados los riesgos locales.
Evidencia	Todavía no iniciado ni autorizado.
Pendiente o límite	Requiere HTTPS, acceso restringido, PostgreSQL aislado, secretos exclusivos, backups, smoke tests y commit identificable.
Lectura del checklist	0/14 ítems marcados

Detalle completo del bloque

Tareas del bloque

PEND.

Crear subdominio de staging separado.

PEND.

Restringir acceso al equipo mediante autenticación adicional.

PEND.

Configurar HTTPS.

PEND.

Separar procesos de web, bot y scheduler.

PEND.

Crear PostgreSQL exclusivo de staging.

PEND.

No exponer el puerto 5432 públicamente.

PEND.

Usar secretos exclusivos de staging y fuera de Git.

PEND.

Crear datos ficticios reproducibles.

PEND.

Identificar cada versión desplegada con su commit.

PEND.

Ejecutar migraciones, smoke tests y QA después de cada despliegue.

PEND.

Respaldar antes de migraciones o cambios de esquema.

Puerta de salida

PEND.

El equipo puede probar el sistema sin acceso público abierto.

PEND.

Staging puede reiniciarse sin afectar la demo final.

PEND.

Los fallos del servidor pueden reproducirse localmente.

Bloque 9

Demo final cerrada y cierre

PENDIENTE

Prioridad: No indicada

Propósito y resultado	Preparar una demo final cerrada, reproducible y separada de staging, además de un cierre ordenado del servicio temporal.
Evidencia	Todavía no iniciado.
Pendiente o límite	Incluye cuentas ficticias, restauración, rollback, guion, evidencias, fecha de apagado y revocación de accesos.
Lectura del checklist	0/13 ítems marcados

Detalle completo del bloque

Tareas del bloque

PEND.	Crear ambiente de demo separado de staging.
PEND.	Deshabilitar registro abierto o utilizar cuentas ficticias precreadas.
PEND.	Limitar acceso a equipo, docentes e invitados.
PEND.	Etiquetar el commit candidato a demostración.
PEND.	Ejecutar suite, lint, build y smoke test final.
PEND.	Probar restauración de PostgreSQL.
PEND.	Preparar rollback a la versión anterior.
PEND.	Preparar guion y datos para la demostración.
PEND.	Documentar instalación, variables, migraciones y limitaciones.
PEND.	Archivar evidencias de pruebas y versión final.
PEND.	Definir fecha de apagado del servicio temporal.
PEND.	Al terminar, exportar lo necesario, revocar accesos y apagar contenedores.
PEND.	Confirmar que no queden secretos o servicios olvidados en el VPS.

13. Definición global de terminado

Estas condiciones no se consideran cumplidas hasta completar staging, demo y cierre. Sirven como aceptación final del proyecto.

PEND.	Un clon limpio puede instalarse sin una base heredada.
PEND.	No hay migraciones destructivas en el flujo normal.
PEND.	PostgreSQL es la fuente principal de datos.
PEND.	Autenticación, ownership, tareas y fechas tienen pruebas.
PEND.	Web, Telegram, Google e IA siguen reglas coherentes.
PEND.	Staging y demo utilizan únicamente usuarios ficticios.
PEND.	La demo final es reproducible desde un commit identificado.
PEND.	El proyecto puede archivarse y apagarse de manera ordenada.

Riesgos transversales vigentes

Riesgo	Tratamiento requerido
Estado del bot y rate limiting en memoria	Definir persistencia o limitar explícitamente a una instancia.
Proveedor de IA disperso	Unificar modelo, prompts, timeout, fallback, fuente y telemetría.
Dashboard monolítico	Extraer componentes y separar presentación, API y estado.
Recordatorios incompletos	Definir reminder_at, zona horaria, reprogramación e idempotencia.
Netcup prematuro	No desplegar antes de cerrar Bloques 5 a 7 y preparar backup/rollback.

Próximas diez acciones recomendadas

1. Unificar proveedor, modelo, prompts, timeouts y fallbacks de IA.
2. Registrar la fuente real de cada recomendación.
3. Separar recomendaciones de IA de las subtareas persistidas.
4. Definir procesos web, bot y scheduler y su estado tras reinicios.
5. Modularizar dashboard.astro sin cambiar comportamiento.
6. Retirar Supabase y nombres Gemini heredados que ya no apliquen.
7. Completar el modelo de recordatorios y reminder_at.
8. Cerrar Google Calendar y renovación de tokens.
9. Ejecutar la matriz de QA móvil, escritorio, teclado y accesibilidad.
10. Solo entonces preparar el checklist de staging Netcup.

14. Evidencias y documentos de soporte

La evidencia se distribuye en documentos especializados. Este PDF es el índice maestro y no sustituye logs, capturas o resultados de CI.

Documento	Contenido
README.md	Arquitectura, instalación, Docker, variables y pruebas.
AGENTS.md	Reglas operativas compartidas para agentes de IA.
CLAUDE.md	Puente de Claude Code hacia AGENTS.md.
docs/TODO_DESARROLLO.md	Checklist canónico de los Bloques 0 a 9.
docs/LINEA_BASE_BLOQUE_0.md	Entorno inicial, riesgos y protección de datos.
docs/CIERRE_BLOQUE_1.md	Seguridad crítica y dependencias.
docs/CIERRE_BLOQUE_2.md	Esquema PostgreSQL y QA funcional.
docs/CIERRE_BLOQUE_3.md	Suite, cobertura y GitHub Actions.
docs/BLOQUE_4_LINEA_BASE.md	Copia SQLite, conteos y ensayos de importación.
docs/CIERRE_BLOQUE_4.md	Docker, PostgreSQL, smoke test y rollback.
docs/QA_IA_LOCAL.md	Prueba real de z.ai y control de alucinaciones.
docs/QA_TELEGRAM_LOCAL.md	Vinculación, tareas, IA y notificaciones.
resumen_avance_y_pendientes_novatareas.pdf	Resumen ejecutivo corto del cierre del Bloque 4.
plan_mejora_y_migracion_novatareas.pdf	Plan original previo a la ejecución; conservar como histórico.

Control de cambios

Actualizar este PDF cuando cambie la puerta de un bloque, el runtime, los comandos de operación o el alcance de Netcup. El estado de Git y los documentos del repositorio deben verificarse nuevamente antes de cada edición.

VEREDICTO

Continuar con el Bloque 5. Netcup no se inicia todavía.